

SafeGuardian AI: An Agentic Women’s Safety and Intelligent SOS Response Platform

B. Akhil Varma

Department of Computer Science and Engineering
KLH Deemed to be University, Bowrampet
Hyderabad, India
Email: abc@example.com

K. Vamsidhar

Department of Computer Science and Engineering
KLH Deemed to be University, Bowrampet
Hyderabad, India
Email: abc@example.com

K.V.S. Jaswanth

Department of Computer Science and Engineering
KLH Deemed to be University, Bowrampet
Hyderabad, India
Email: abc@example.com

Abstract—Women’s safety systems require rapid emergency response with minimum user interaction. Existing safety solutions differ in the degree to which emergency detection, location acquisition, artificial intelligence analysis, trusted contact management, and notification services are integrated into a single workflow. This paper presents SafeGuardian AI, an integrated emergency response platform that combines one-click SOS activation, browser-based location capture, trusted contact management, artificial-intelligence-assisted threat assessment, and emergency notification services. The system uses a web frontend, Spring Boot backend, PostgreSQL database, and FastAPI-based artificial intelligence service. The proposed architecture is inspired by the emergency response concept of the Rakshak framework while adopting a service-oriented architecture. The proposed extension considers contextual information from location, voice, and image evidence for multimodal emergency assessment. The paper defines an evaluation methodology using accuracy, precision, recall, and F1-score for model assessment together with system-level measures such as SOS processing success, location capture success, processing latency, and notification reliability. Quantitative model results are intentionally reported only after evaluation on a labeled dataset.

Index Terms—women’s safety, artificial intelligence, SOS, emergency response, multimodal assessment, computer vision, voice analysis, Spring Boot, FastAPI

I. INTRODUCTION

Women’s safety applications have increasingly incorporated digital technologies to reduce emergency response time and provide rapid communication with trusted individuals. Conventional systems often depend on a manually activated panic button or require multiple user actions before an emergency alert is transmitted. During a real emergency, the ability to perform several actions may be reduced because of stress, physical restriction, lack of time, or environmental constraints.

The Rakshak framework presents a multi-trigger emergency-response approach that considers voice instructions, eye-blink patterns, and tap-based triggers for automatic emergency detection and alert generation [1]. The framework also emphasizes rapid communication of the user’s location to registered emergency contacts.

SafeGuardian AI adopts the broader objective of minimizing emergency-response interaction while implementing a different service-oriented architecture. The proposed system uses one-click SOS activation as the primary emergency trigger and coordinates location capture, artificial intelligence assessment, trusted contact retrieval, and emergency notification through a backend orchestration layer.

The main workflow of SafeGuardian AI is:

$$SOS \rightarrow Location \rightarrow AI\ Assessment \rightarrow Contacts \rightarrow Notification \quad (1)$$

The objective is not simply to provide an SOS button, but to create an integrated pipeline in which individual services communicate through well-defined application programming interfaces.

The major contributions of this work are as follows:

- A service-oriented emergency-response architecture for women’s safety.
- Integration of SOS activation, browser location acquisition, trusted-contact management, and AI processing.
- A multimodal assessment concept using location, voice, and image evidence.
- A distinction between AI model performance and external notification-service reliability.
- A quantitative evaluation methodology based on accuracy, precision, recall, and F1-score.

II. RELATED WORK

A. Rakshak Framework

Patil *et al.* proposed Rakshak, a multi-layered intelligent framework for proactive emergency response in women’s safety systems [1]. The system addresses limitations of safety applications that rely primarily on manual activation and continuous connectivity.

Rakshak describes a multi-trigger approach incorporating voice instructions, eye-blink patterns, and tap-based triggers.

When distress is detected, the system can generate an emergency alert and communicate the user's location to registered emergency contacts [1].

The work therefore establishes a useful emergency-response concept in which distress detection and alert generation are connected into an integrated workflow.

B. Research Motivation

SafeGuardian AI uses a different implementation scope. Instead of reproducing every trigger mechanism in Rakshak, the proposed system concentrates on service integration and contextual emergency assessment.

The comparison is therefore primarily architectural and functional. A numerical comparison of model performance would require both systems to be evaluated using the same dataset, test conditions, and evaluation protocol.

III. PROBLEM STATEMENT

There are many websites and mobile applications for women's safety, ranging from basic to advanced solutions. However, the level of integration between different emergency services varies. During an emergency, users may still need to perform multiple actions, which can introduce delays.

Therefore, an integrated emergency-response system is required to coordinate essential services while minimizing the amount of interaction required from the user. Such a system should be capable of collecting emergency context, processing that information, retrieving trusted contacts, and initiating emergency communication through a unified workflow.

IV. OBJECTIVES

The objectives of SafeGuardian AI are:

- 1) To provide a minimal-interaction SOS mechanism.
- 2) To capture the user's current location through browser geolocation.
- 3) To store and retrieve trusted emergency contacts.
- 4) To provide an AI service for emergency context assessment.
- 5) To support voice and image evidence as multimodal inputs.
- 6) To coordinate emergency operations through REST APIs.
- 7) To initiate emergency notification to trusted contacts.
- 8) To explicitly report failed or unavailable services instead of presenting false success states.

V. SYSTEM ARCHITECTURE

The proposed architecture separates user interaction, application orchestration, data persistence, AI processing, and notification delivery.

A. Frontend Layer

The frontend provides user interaction, login and registration interfaces, trusted-contact management, and SOS initiation. Browser geolocation functionality is used to obtain the current position during SOS activation.

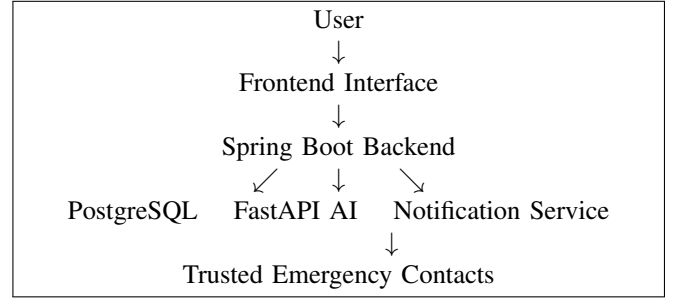


Fig. 1. High-level SafeGuardian AI architecture.

B. Backend Layer

Spring Boot acts as the primary orchestration component. It receives REST requests from the frontend, retrieves trusted contacts from PostgreSQL, communicates with AI services, and initiates notification processing.

C. Database Layer

PostgreSQL stores trusted-contact information and application data. The database allows the SOS workflow to retrieve a user's registered emergency contacts during an emergency.

D. AI Layer

The AI layer is designed to support multiple forms of evidence. The planned technology stack includes YOLOv11 and OpenCV for image processing, MediaPipe for pose-related information, Whisper for speech transcription, and PyTorch for model execution and machine-learning support.

E. Notification Layer

The notification layer is designed to communicate emergency information to trusted contacts. The current implementation uses Twilio WhatsApp API integration. Provider-level restrictions are treated separately from AI model performance.

VI. PROPOSED METHODOLOGY

The SafeGuardian AI methodology is summarized as:

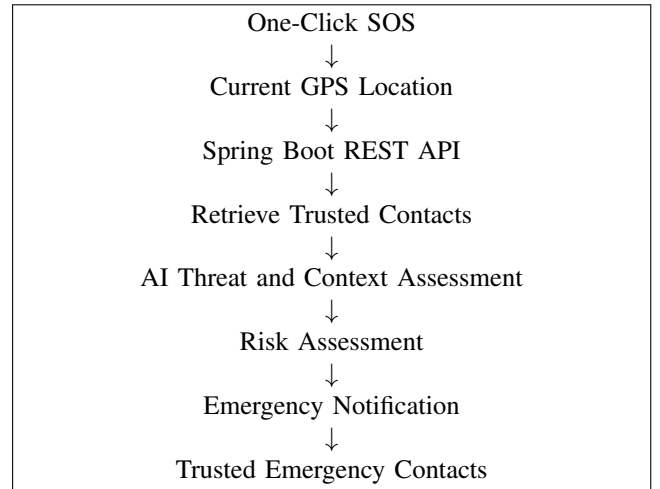


Fig. 2. SafeGuardian AI emergency-response methodology.

A. SOS Activation

The user initiates an emergency using the SOS interface. The frontend sends an SOS request to the Spring Boot backend.

B. Location Acquisition

The browser geolocation API is used to acquire the user's current location. The resulting latitude and longitude are included in the emergency request.

C. Trusted Contact Retrieval

The backend retrieves saved trusted contacts from PostgreSQL. The prototype allows multiple trusted contacts to be maintained, subject to the application's configured limit.

D. AI Assessment

The AI service receives available contextual information and may process voice or image evidence. The objective is to determine whether the available evidence indicates a potentially dangerous situation.

E. Emergency Notification

After processing the emergency request, the notification layer attempts to communicate the emergency event and location to registered contacts.

VII. MULTIMODAL AI DESIGN

The proposed novelty of SafeGuardian AI is context-based multimodal emergency assessment and evidence sharing.

The concept is represented as:

$$Location + Voice + Image \rightarrow Context Assessment \rightarrow Emergency Response \quad (2)$$

TABLE I
PROPOSED MULTIMODAL AI COMPONENTS

Input	Technology / Purpose
Location	Browser Geolocation API for contextual location information
Voice	Whisper for speech transcription and voice evidence processing
Image	YOLOv11 and OpenCV for visual analysis
Pose	MediaPipe for landmark and pose information
Fusion	PyTorch-supported processing for feature integration

The multimodal architecture is intended to provide additional context rather than depend solely on a single emergency signal.

It should be noted that the presence of a technology in the architecture does not by itself demonstrate quantitative model performance. Such performance must be established experimentally.

VIII. IMPLEMENTATION

The implementation uses the technology stack shown in Table II.

TABLE II
SAFEGUARDIAN AI TECHNOLOGY STACK

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Spring Boot
Database	PostgreSQL
AI API	FastAPI
Computer Vision	YOLOv11, OpenCV
Pose Analysis	MediaPipe
Speech	Whisper
ML Runtime	PyTorch
Notification	Twilio WhatsApp API
Communication	REST APIs

The frontend communicates with Spring Boot through REST APIs. The backend performs application orchestration and communicates with PostgreSQL and the AI service.

IX. MODEL EVALUATION

Quantitative evaluation is necessary to determine whether an AI model performs reliably on previously unseen data. In the context of SafeGuardian AI, model evaluation should be performed using a labeled dataset containing emergency and non-emergency samples.

For example, samples can be divided into two classes:

- Emergency or distress.
- Normal or non-emergency.

A possible experimental split is:

TABLE III
PROPOSED DATASET SPLIT

Dataset Portion	Percentage
Training	70%
Validation	15%
Testing	15%

The exact dataset size and distribution should be reported after the experimental dataset has been constructed.

A. Confusion Matrix

For binary classification, four quantities are used:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

TABLE IV
BINARY CLASSIFICATION CONFUSION MATRIX

	Predicted Emergency	Predicted Normal
Actual Emergency	TP	FN
Actual Normal	FP	TN

B. Accuracy

Accuracy represents the percentage of all test samples that are classified correctly.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

C. Precision

Precision represents the proportion of predicted emergency cases that are actually emergency cases.

$$Precision = \frac{TP}{TP + FP} \quad (4)$$

Precision is important because a large number of false alarms may reduce user trust in the emergency system.

D. Recall

Recall measures the proportion of actual emergency cases detected by the model.

$$Recall = \frac{TP}{TP + FN} \quad (5)$$

For an emergency-response system, recall is particularly important because false negatives correspond to emergency situations that were not identified.

E. F1-Score

The F1-score provides a combined measure of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (6)$$

X. MODEL EVALUATION RESULTS

The following table defines the results that should be reported after the labeled evaluation dataset is tested.

TABLE V
SAFEGUARDIAN AI MODEL EVALUATION

Metric	Result
Accuracy	To be measured
Precision	To be measured
Recall	To be measured
F1-Score	To be measured
False Positive Rate	To be measured
False Negative Rate	To be measured

No numerical performance value is assigned in this paper without an experimental evaluation. Model confidence values generated during inference should not be presented as accuracy or precision.

XI. SYSTEM-LEVEL EVALUATION

AI model performance alone does not describe the reliability of a complete emergency-response system. Therefore, SafeGuardian AI is evaluated at the system level as well.

This distinction is important because a failure in an external notification provider does not constitute an AI model failure.

TABLE VI
SYSTEM-LEVEL EVALUATION METRICS

Metric	Measurement
SOS Activation Success	Successful SOS requests divided by total SOS attempts
GPS Capture Success	Successful location captures divided by location attempts
AI Processing Success	Completed AI assessments divided by AI requests
Contact Retrieval Success	Successful trusted-contact retrieval operations
Notification Success	Successful notification deliveries divided by attempts
End-to-End Latency	Time from SOS activation to notification attempt
Failure Handling	Correct FAILED or NOT_CONFIGURED state reporting

TABLE VII
CURRENT SAFE GUARDIAN AI PROTOTYPE VALIDATION

Workflow Stage	Observed Status
SOS Activation	Successful
GPS Location Capture	Successful
AI Assessment	Successful
Trusted Contact Retrieval	Successful
Notification Request	Initiated
WhatsApp Delivery	Not Validated

XII. CURRENT PROTOTYPE VALIDATION

During local prototype testing, the major stages of the SOS workflow were observed.

The current prototype successfully executes the SOS workflow through the location, AI assessment, and trusted-contact processing stages.

During the present integration testing, the external Twilio trial configuration rejected the WhatsApp request. Therefore, notification delivery should be reported as an integration limitation rather than as an AI accuracy problem.

XIII. COMPARISON WITH RAKSHAK

Rakshak and SafeGuardian AI address the same broad problem of improving emergency response, but they differ in their implementation and architectural emphasis.

Rakshak describes a multi-trigger system involving voice instructions, eye-blink patterns, and tap-based emergency triggers, followed by emergency alerts and location communication [1].

SafeGuardian AI uses a one-click SOS workflow and emphasizes service integration through a web-based architecture.

The comparison is functional and architectural rather than a ranking. A direct accuracy comparison would require both systems to be tested on the same dataset under identical experimental conditions.

TABLE VIII
FUNCTIONAL COMPARISON OF RAKSHAK AND SAFEGUARDIAN AI

Feature	Rakshak	SafeGuardian AI
Primary Trigger	Multi-trigger	One-click SOS
Voice Trigger	Yes	Proposed AI input
Eye-blink Trigger	Yes	Not primary trigger
Tap Trigger	Yes	Not primary trigger
Location	Yes	Yes
Emergency Contacts	Yes	Yes
AI Processing	Yes	Yes
Voice Evidence	Yes	Proposed
Image Evidence	Not established from publisher abstract	Proposed
PostgreSQL	Not specified in publisher abstract	Yes
FastAPI Service	Not specified in publisher abstract	Yes
Spring Boot	Not specified in publisher abstract	Yes
REST-Oriented Integration	Not the primary comparison point	Central design
WhatsApp API	Not established from publisher abstract	Integrated but currently constrained

XIV. PROPOSED NOVELTY

The proposed novelty of SafeGuardian AI is:

Context-Based Multimodal Emergency Assessment and Evidence Sharing.

The concept combines multiple emergency-context sources:

$$Location + Voice + Image \rightarrow Context Assessment \rightarrow Emergency Response \quad (7)$$

The intention is to provide the emergency-response process with additional context rather than relying solely on the initial SOS action.

This novelty is presented as a proposed extension and requires further experimental validation.

XV. DISCUSSION

The SafeGuardian AI architecture demonstrates how an emergency-response application can be decomposed into independent services while maintaining a coordinated workflow.

The separation between frontend, backend, AI service, database, and notification service provides several advantages. Individual components can be tested independently, failures can be isolated, and future services can be added without redesigning the entire application.

The architecture also highlights an important distinction between model-level and system-level evaluation. A system may contain an AI model with good classification performance while still failing to deliver an emergency message because of network, account, provider, or configuration constraints.

Therefore, a complete evaluation should report both AI metrics and system reliability metrics.

XVI. LIMITATIONS

The current work has several limitations.

First, quantitative model evaluation requires a labeled dataset containing sufficient examples of both emergency and non-emergency conditions.

Second, the proposed multimodal approach requires evaluation under diverse lighting, background, camera angle, speech, and environmental conditions.

Third, external notification services may impose account-level and API-level restrictions. During the current prototype testing, the Twilio trial configuration did not permit successful WhatsApp notification delivery.

Fourth, public evidence sharing requires a publicly accessible endpoint and cannot be fully validated using a local-only deployment.

Finally, a fair numerical comparison with Rakshak cannot be established without reproducing or accessing a comparable benchmark dataset and evaluation methodology.

XVII. FUTURE WORK

Future work will focus on the following areas:

- 1) Construction of a labeled emergency and non-emergency dataset.
- 2) Quantitative evaluation of the multimodal AI pipeline.
- 3) Training and validation of dedicated threat-classification models.
- 4) Voice-based automatic emergency detection.
- 5) Silent emergency activation using gestures or physiological cues.
- 6) Offline or fallback communication mechanisms.
- 7) SMS fallback when internet-based messaging is unavailable.
- 8) Publicly accessible evidence storage and sharing.
- 9) Continuous location updates during active emergencies.
- 10) Explainable AI outputs indicating the factors contributing to risk assessment.
- 11) Larger-scale evaluation under realistic environmental conditions.

XVIII. CONCLUSION

SafeGuardian AI presents a service-oriented architecture for intelligent women's safety and emergency response. The proposed platform integrates one-click SOS activation, location acquisition, trusted-contact management, AI processing, and emergency notification into a coordinated workflow.

The design is conceptually related to the emergency-response objective of Rakshak while implementing a different architectural approach. Rakshak emphasizes multi-trigger emergency detection through voice instructions, eye-blink patterns, and tap-based triggers [1]. SafeGuardian AI instead concentrates on a one-click SOS workflow supported by backend service orchestration and proposed multimodal contextual assessment.

The proposed multimodal component combines location, voice, and image information for contextual emergency analysis. However, quantitative model performance must be determined experimentally using a labeled evaluation dataset. Accuracy, precision, recall, and F1-score should therefore only be reported after the corresponding confusion-matrix values have been obtained.

The current prototype demonstrates successful execution of the SOS workflow through location capture, AI assessment, and trusted-contact retrieval. Notification delivery remains dependent on external provider configuration. The architecture provides a foundation for future research into multimodal, explainable, and integrated emergency-response systems.

ACKNOWLEDGMENT

The authors would like to thank the Department of Computer Science and Engineering, KLU University, for providing the academic environment and infrastructure required for this work.

REFERENCES

- [1] S. Patil, S. Kanawade, Y. Karnawat, K. Deshmukh, S. Chaudhari, and C. Karodpati, "Rakshak: A Multi-Layered Intelligent Framework for Offline-Capable Proactive Emergency Response in Women's Safety Systems," in *Proceedings of the International Conference on Advances in Computing Technology and Artificial Intelligence (COMPUTATIA 2026)*, pp. 673–687, 2026, doi: 10.2991/978-94-6239-713-2_50.
- [2] VMware, "Spring Boot Documentation," Spring.io.
- [3] S. Ramirez, "FastAPI Documentation," FastAPI.
- [4] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [6] C. Lugaesi *et al.*, "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019.